

■ BetCopa 2026

Sistema de Apostas da Copa do Mundo

Guia de Apresentação

Trabalho T2 — Paradigmas de Linguagens de Programação (PLP)

Comparação entre Programação Orientada a Objetos, Funcional e Imperativa

1. Visão Geral do Projeto

O **BetCopa 2026** é um sistema web de apostas esportivas para os jogos da Copa do Mundo 2026 (EUA, Canadá e México). O usuário se cadastra, recebe um saldo virtual e aposta nos resultados das partidas. O sistema consome dados **reais e ao vivo** da API pública TheSportsDB — placares, grupos e status das partidas são atualizados automaticamente.

Tecnologias

- **HTML, CSS e JavaScript puro** (sem frameworks) — roda em qualquer navegador.
- **API TheSportsDB** — dados reais da Copa 2026 (72 jogos, grupos A–L).
- **flagcdn.com** — bandeiras oficiais dos países.

Por que esse tema?

■ ■ **O que dizer:** Escolhemos um sistema de apostas porque ele junta três coisas que mostram bem os paradigmas: **entidades do mundo real** (jogos, usuários, apostas) que pedem Orientação a Objetos; **cálculos e transformações de dados** (estatísticas, ranking, tabela de grupos) que ficam elegantes em estilo Funcional; e um **fluxo de tela** que é naturalmente Imperativo. Assim conseguimos comparar os três no mesmo projeto.

Requisitos do trabalho atendidos

Requisito	Como atendemos
3+ funcionalidades	Apostas, classificação de grupos, ranking, painel admin
Orientação a Objetos	Classes Jogo, Usuario, Aposta, SistemaApostas
Recurso Funcional	map / filter / reduce, funções puras, imutabilidade
Relatório comparativo	Seção 5 deste material
Demonstração prática	Roteiro na seção 4

2. Funcionalidades Principais

■ Apostas

O usuário escolhe time 1, empate ou time 2, define o valor e o sistema calcula o retorno potencial pela odd. O saldo é debitado na hora.

■ Classificação dos Grupos

Tabela de pontos calculada automaticamente a partir dos resultados reais — vitórias, empates, saldo de gols e pontuação.

■ Ranking de Apostadores

Leaderboard ordenado por lucro, com taxa de acerto de cada jogador.

■■ Painel Admin

Permite inserir resultados manualmente; ao salvar, todas as apostas pendentes daquele jogo são resolvidas automaticamente.

■ Dados ao vivo

Integração com a API real: jogos ao vivo, placares e status atualizam sozinhos a cada 60 segundos.

3. Os Paradigmas Utilizados

Este é o **coração da apresentação**. O projeto combina três paradigmas, cada um em um arquivo diferente, o que torna a comparação muito clara.

3.1 — Orientação a Objetos (arquivo: models.js)

Modelamos o mundo real como **objetos** que juntam dados e comportamentos. Cada entidade vira uma classe.

```
class Aposta {  
  constructor(usuario, jogo, escolha, valor) { ... }  
  resolver() { /* a própria aposta sabe se ganhou */ }  
}
```

- **Encapsulamento:** usuario.adicionarAposta() cuida do saldo internamente.
- **Abstração:** jogo.getNomeVencedor() esconde a lógica de comparação.
- **Composição:** uma Aposta contém referências a um Jogo e a um Usuario.

■ ■ **O que dizer:** Usamos OOP para as entidades do domínio. A classe **Aposta**, por exemplo, sabe como se resolver sozinha quando o jogo termina — chamamos `aposta.resolver()` e ela verifica o resultado, paga o usuário se ganhou e atualiza o próprio estado. Cada classe tem uma responsabilidade clara, o que facilita manter o código.

3.2 — Programação Funcional (arquivo: functional.js)

Funções **puras**: dado o mesmo input, sempre retornam o mesmo output, sem alterar nada fora delas. Os dados são transformados encadeando funções.

```
const calcularLucroTotal = apostas =>  
  apostas  
    .filter(a => a.resultado === 'ganhou')  
    .reduce((acc, a) => acc + a.ganho, 0);
```

- **Funções puras:** sem efeitos colaterais, fáceis de testar isoladamente.
- **Imutabilidade:** [...array].sort() cria uma cópia em vez de alterar o original.
- **Higher-order functions:** filter, map e reduce recebem funções como argumento.
- **Arrow functions:** sintaxe enxuta para transformar dados.

■ ■ **O que dizer:** No arquivo `functional.js` todas as funções são puras. Para calcular o lucro de um jogador, por exemplo, encadeamos **filter** e **reduce** — primeiro filtramos só as apostas vencedoras, depois somamos os ganhos. Não tem nenhum laço 'for' nem variável sendo modificada. O código fica quase como uma frase em inglês: 'filtre as ganhas e reduza somando'. A tabela de classificação dos grupos também é feita assim.

3.3 — Programação Imperativa (arquivo: app.js)

O controlador da interface segue o estilo imperativo: comandos passo a passo, estado mutável e fluxo de controle explícito — o paradigma mais familiar.

■ ■ **O que dizer:** O `app.js` é imperativo: ele descreve o passo a passo — 'pegue o jogo, valide o valor, debite o saldo, atualize a tela'. É o jeito mais intuitivo de pensar, mas quando o sistema cresce ele tende a misturar responsabilidades. Por isso isolamos as regras de negócio nas classes e os cálculos nas funções puras.

4. Roteiro da Demonstração Prática

Sequência sugerida para mostrar o sistema funcionando ao vivo. Deixe o servidor já rodando antes de começar (npx serve -p 3000).

Passo 1 — Tela inicial

Mostre o banner '72 jogos carregados da API' e os jogos ao vivo. Destaque que os placares são REAIS, vindos da Copa 2026.

Passo 2 — Login

Entre com a conta demo (demo@copa2026.com / 123456). Mostre o saldo aparecendo no topo.

Passo 3 — Fazer uma aposta

Vá em Apostas, escolha um jogo, selecione um time, digite o valor. Mostre o retorno potencial calculado em tempo real e confirme.

Passo 4 — Grupos

Abra a aba Grupos e mostre a tabela de classificação calculada automaticamente a partir dos resultados — esse cálculo é 100% funcional.

Passo 5 — Admin resolve um jogo

No painel Admin, defina o resultado de um jogo em que você apostou. Mostre que a aposta foi resolvida e o saldo atualizado sozinho.

Passo 6 — Ranking

Finalize mostrando o ranking de apostadores ordenado por lucro.

■■ **O que dizer:** Dica: aposte de propósito em um time e depois, no Admin, marque esse time como vencedor. Assim a plateia vê o saldo subir na hora — é o momento que conecta todos os paradigmas: a classe Aposta se resolve (OOP), as estatísticas recalculam (Funcional) e a tela atualiza (Imperativo).

5. Comparação dos Paradigmas (Relatório)

Respostas diretas às perguntas pedidas no trabalho.

Qual paradigma foi mais fácil de utilizar?

O **Imperativo**, porque é o mais intuitivo — você escreve na ordem em que as coisas acontecem, igual ao raciocínio do dia a dia.

Qual gerou o código mais legível?

O **Funcional**. Expressões como `apostas.filter(...).reduce(...)` são curtas e quase se leem como linguagem natural, sem variáveis de controle e sem laços.

Quais vantagens e limitações foram observadas?

OOP: ótima organização e reúso, mas o estado mutável compartilhado pode gerar bugs difíceis de rastrear. **Funcional**: previsível e fácil de testar, mas tem curva de aprendizado e pode custar mais memória em listas enormes. **Imperativo**: simples de começar, mas vira 'código espaguete' quando cresce.

Em quais situações cada um é mais indicado?

OOP para modelar entidades complexas de um domínio (jogos, usuários). **Funcional** para transformar e analisar dados (estatísticas, relatórios, pipelines). **Imperativo** para scripts diretos e controle de fluxo de tela.

Resumo comparativo

Critério	OOP	Funcional	Imperativo
Onde no projeto	models.js	functional.js	app.js
Foco	Entidades	Transformar dados	Fluxo de tela
Estado	Mutável	Imutável	Mutável
Legibilidade	Boa	Excelente	Boa (cai ao crescer)
Facilidade de teste	Média	Alta	Baixa

Conclusão: nenhum paradigma é 'melhor' em absoluto. O projeto mostra que o ideal é **combiná-los** — usar cada um onde ele brilha. Foi exatamente o que fizemos no BetCopa 2026.